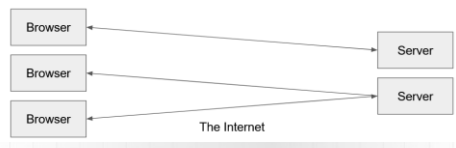


Cybersecurity
Web security



WWW

- **Web (World Wide Web):** A collection of data and services
 - Data and services are provided by **web servers**
 - Data and services are accessed using **web browsers** (e.g. Chrome, Firefox)
- **The web is not the Internet**
 - The Internet describes *how* data is transported between servers and browsers



Cybersecurity
Goals of web security

- Safely browse the web
 - Visit a variety of web sites without incurring harm
 - Integrity: Site A cannot compromise session at Site B
 - Confidentiality: no information stolen from your device
- Support secure web apps
 - Apps provided over the web can have same security properties as stand-alone applications
- Support secure mobile apps
 - Web protocols and web content standards are used as back end of many mobile apps

Cybersecurity
Web Threat Models

- **Web attacker**
 - Controls attacker.com
 - Can obtain SSL/TLS certificate for attacker.com
 - User visits attacker.com
 - Or: runs attacker's Facebook app, etc.
- **Network attacker**
 - Passive: Wireless eavesdropper
 - Active: Evil router, DNS poisoning
- **Malware attacker**
 - Attacker escapes browser isolation mechanisms and run separately under control of OS

Cybersecurity
Malware attacker

- Browsers may contain exploitable bugs
 - Often enable remote code execution by web sites
 - 900K Users Compromised: Chrome Extensions Steal ChatGPT and DeepSeek Conversations
 - Malicious Chrome Extensions "Phantom Shuttle" Masquerade as a VPN to Intercept Traffic and Exfiltrate Credentials
 - ...
 - Modern reality (Safe Browsing):
 - Google Safe Browsing examines billions of URLs per day and flags unsafe sites.
- Even if browsers were bug-free, still lots of vulnerabilities associated with the web
 - OWASP top 10: XSS, SQLi, CSRF, ...

Problems

- Http
- Rendering content
- Isolation
- Cookies

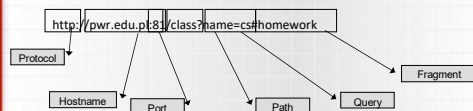
URL

URL

- **URL (Uniform Resource Locator):** A string that uniquely identifies one piece of data on the web
 - A type of URI (Uniform Resource Identifier)
- URLs are designed to contain printable, human-readable characters (ASCII)
 - What if we want to include non-printable characters in the URL?
- Recall: URLs have special characters (?, #, /)
 - What if we want to use a special character in the URL?
- Solution: URL encoding
 - Notation: Percent sign (%) followed by the hexadecimal value of the character
 - Example: %20 = ' ' (spacebar)
 - Example: %23 = '#' (hash sign)
 - Example: %50 = 'P' (printable characters can be encoded too!)
- Security issues: makes scanning for malicious URLs harder
 - Suppose you want to block all requests to the path `/etc/passwd`
 - What if an attacker makes a request to `%2F%65%74%63%2F%70%61%73%77%64?`
 - We'll study this issue more later

URLs

- Global identifiers of network-retrievable documents
- Example:



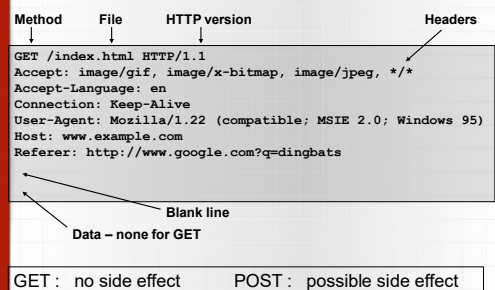
- Special characters are encoded as hex:
 - %0A = newline
 - %20 or + = space, %2B = + (special exception)

HTTP

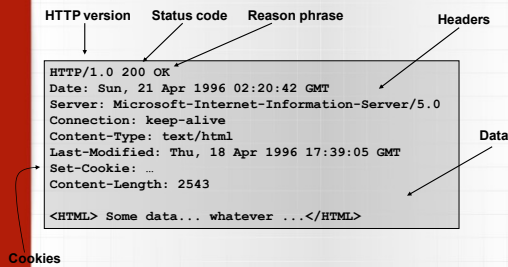
HTTP

- URLs: How do we uniquely identify a piece of data on the web?
- **HTTP: How do web browsers communicate with web servers?**
- Data on a webpage can contain:
 - **HTML:** A markup language for creating webpages
 - **CSS:** A style sheet language for defining the appearance of webpages
 - **JavaScript:** A programming language for running code in the web browser
- **HTTP (Hypertext Transfer Protocol):** A protocol used to request and retrieve data from a web server
- **HTTPS:** A secure version of HTTP
 - Uses cryptography to secure data usually TLS
- HTTP is a request-response model
 - The web browser sends a **request** to the web server
 - The web server processes the request and sends a **response** to the web browser

HTTP Request



HTTP Response



Rendering Content

Rendering and events

- Basic browser execution model
 - Each browser window or frame
 - Loads content
 - Renders it
 - Processes HTML and scripts to display page
 - May involve images, subframes, etc.
 - Responds to events
- Events can be
 - User actions: OnClick, OnMouseover
 - Rendering: OnLoad, OnBeforeUnload
 - Timing: setTimeout, clearTimeout

Rendering

- Process of displaying (rendering) a webpage in a web browser:
 - The browser receives HTML, CSS, and JavaScript from the server
 - HTML and CSS are parsed into a **DOM (Document Object Model)**
 - JavaScript is interpreted and executed, possibly modifying the DOM
 - The painter uses the DOM to draw the webpage
- **DOM (Document Object Model):** Cross-platform model for representing and interacting with objects in HTML
 - A tree of nodes
 - Each node has a tag, attributes, and child nodes

Document Object Model (DOM)

- Object-oriented interface used to read and write docs
 - web page in HTML is structured data
 - DOM provides representation of this data structure
- Examples
 - **Properties:** document.alinkColor, document.URL, document.forms[], document.links[], document.anchors[]
 - **Methods:** document.write(document.referrer)
- Browser Object Model (BOM)
 - window, document, frames[], history, location, navigator (type and version of browser)

Rendering Content – Think like an attacker

Basic web functionality HTML Image Tags

```
<html>
...
<p> ... </p>
...

...
</html>
```

Displays this nice picture →
Security issues?



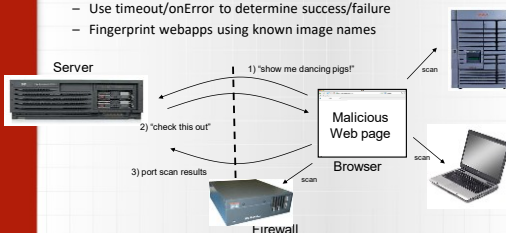
Security consequences Image tag security issues

- Communicate with other sites
 - ``
- Hide resulting image
 - ``
- Spoof other sites
 - Add logos that fool a user



Security consequence Port scanning behind firewall

- JavaScript can:
 - Request images from internal IP addresses
 - Example: ``
 - Use timeout/onError to determine success/failure
 - Fingerprint webapps using known image names



WEB Security

Isolation

Risks

- Risk #1: Web servers should be protected from unauthorized access
 - Example: An attacker should not be able to hack into `google.com` and provide malicious search results to users
- Protection: Server-side security
 - Example: Protect the server computer from buffer overflow attacks

Risks

- Risk #2: A malicious website should not be able to damage our computer
 - Example: Visiting `evil.com` should not infect our computer with malware
 - Example: If we visit `evil.com`, the attacker who owns `evil.com` should not be able to read/write files on our computer
- Protection: Sandboxing
 - JavaScript is not allowed to access files on our computer
 - Privilege separation, least privilege
 - Browsers are carefully written to avoid exploiting the browser's code (e.g. write the browser in a memory-safe language)

Risks

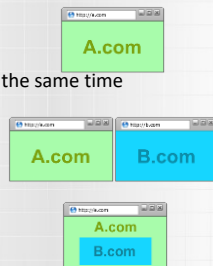
- Risk #3: A malicious website should not be able to tamper with our information or interactions on other websites
 - Example: If we visit `evil.com`, the attacker who owns `evil.com` should not be able to read our emails or buy things with our Amazon account
- Protection: Same-origin policy
 - The web browser prevents a website from accessing other *unrelated* websites

Analogy

- Operating System
 - Primitives
 - System calls
 - Processes
 - Disk
 - Principals: Users
 - Discretionary access control
 - Vulnerabilities
 - Buffer overflow
 - Root exploit
- Web browser
 - Primitives
 - Document object model
 - Frames
 - Cookies / localStorage
 - Principals: "Origins"
 - Mandatory access control
 - Vulnerabilities
 - Cross-site scripting
 - Cross-site request forgery
 - Cache history attacks
 - ...

Policy Goals

- Safe to visit an evil web site
- Safe to visit two pages at the same time
 - Address bar distinguishes them
- Allow safe delegation



SOP

- **Same-origin policy:**
- A rule that prevents one website from tampering with other *unrelated* websites
 - Enforced by the web browser
 - Prevents a malicious website from tampering with behavior on other websites

SOP

- Every webpage has an **origin** defined by its URL with three parts:
 - **Protocol:** The protocol in the URL
 - **Domain:** The domain in the URL's location
 - **Port:** The port in the URL's location
 - If no port is specified, the default is 80 for HTTP and 443 for HTTPS
- `https://pwr.edu.pl:443/assets/lock.PNG`
- `http://portal.pwr.edu.pl/assets/images/404.png`
- 80 (default port)

SOP

- Two webpages have the same origin **if and only if** the protocol, domain, and port of the URL all match exactly
 - Think string matching: The protocol, domain, and port strings must be equal

First domain	Second domain	Same origin?
<code>http://pwr.edu.pl</code>	<code>https://pwr.edu.pl</code>	Protocol mismatch <code>http</code> $\neq$ <code>https</code>
<code>http://pwr.edu.pl</code>	<code>http://portal.pwr.edu.pl</code>	Domain mismatch <code>portal.pwr.edu.pl</code> $\neq$ <code>pwr.edu.pl</code>
<code>http://pwr.edu.pl[:80]</code>	<code>http://pwr.edu.pl:8000</code>	Port mismatch <code>80</code> $\neq$ <code>8000</code>

SOP

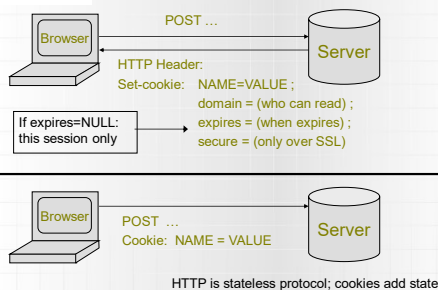
- Two websites with different origins cannot interact with each other
 - Example: If `pwr.edu.pl` embeds `google.com`, the inner frame cannot interact with the outer frame, and the outer frame cannot interact with the inner-frame
- Exception: JavaScript runs with the origin of the page that loads it
 - Example: If `pwr.edu.pl` fetches JavaScript from `google.com`, the JavaScript has the origin of `pwr.edu.pl`
 - Intuition: `pwr.edu.pl` has "copy-pasted" JavaScript onto its webpage
- Exception: Websites can fetch and display images from other origins
 - However, the website only knows about the image's size and dimensions (cannot actually manipulate the image)
- Exception: Websites can agree to allow some limited sharing
 - Cross-origin resource sharing (CORS)
 - The `postMessage` function in JavaScript

Cookies: client state

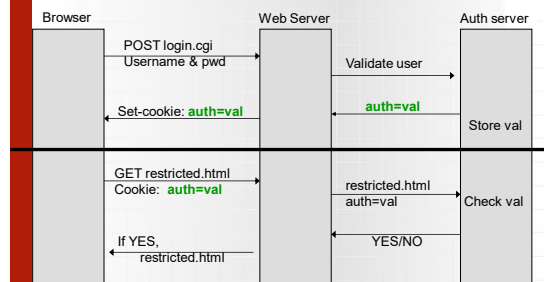
60

Used to store state on user's machine

Cookies



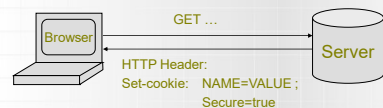
Cookie authentication



Cookie Security Policy

- Uses:
 - User authentication
 - Personalization
 - User tracking: e.g. Doubleclick (3rd party cookies)
- Origin is the tuple `<domain, path>`
 - Can set cookies valid across a domain suffix

Secure Cookies



- Provides confidentiality against network attacker
 - Browser will only send cookie back over HTTPS
- ... but no integrity
 - If the site is reachable over HTTP, an active attacker can try to set/shadow cookies (cookie tossing) or force downgrade.
 - Mitigation: HSTS + redirect HTTP→HTTPS, plus cookie prefixes (`__Host/` `Secure=`) and `SameSite`.
 - Risk example: session fixation (user ends up in attacker-controlled session).

- Cookie sent over HTTP(s), but not accessible to scripts
 - cannot be read via `document.cookie`
 - Helps prevent cookie theft via XSS



UNIVERSITY OF TORONTO

